Bondi MDP — Arquitectura técnica y plan de despliegue

> *Documento de referencia técnica del proyecto. Cubre el motor, la matemática que lo sostiene y la estrategia de despliegue por PRs incrementales.*

---

1. Filosofía

La aplicación está construida sobre **catálogo estático + motor propio + GPS crowdsourced**, con la API municipal como complemento (no como dependencia). Esto significa:

- El recorrido y los itinerarios funcionan **sin internet hacia la muni**.
- Los arribos en vivo se calculan con la **velocidad media real del bondi** cuando hay alguien compartiendo viaje por Telegram, y caen a un fallback basado en velocidad urbana promedio cuando no la hay.
- La API muni se consulta solo para arribos oficiales (cuando responde) y para enriquecer la lista de paradas.

---

2. Catálogo de datos

2.1 Paradas y líneas (API muni)

Generadas por `scripts/build-stops.ts` desde fixtures de la API municipal:

- `RecuperarParadasConBanderaYDestinoPorLinea` → líneas con sus banderas (sentidos) y la lista ordenada de paradas por bandera.
- Calles + intersecciones → nombre legible `"Calle X y Calle Y"`.
- Deduplica paradas por coordenadas: si dos líneas frenan en la misma esquina, hay un solo `Stop` con `lineas: ["541","573"]`.

Salida:

- `lib/static/lines.json` — `Line[]` con `codigo`, `descripcion`, `paradas`, `banderas`.
- `lib/static/stops.json` — `Stop[]` con `id`, `nombre`, `lat`, `lng`, `lineas[]`, `banderas[]`.

2.2 Líneas manuales (GeoJSON)

Algunas líneas no están en la API muni (caso 221 Serena ↔ Mar Chiquita). Para esas:

1. Tenemos el recorrido como GeoJSON `LineString` en `/public` (uno por bandera).
2. `scripts/build-manual-stops.mjs` samplea cada `LineString` cada ~400 m (haversine) y genera "paradas sintéticas".
3. Se persisten en `lib/static/manual.json` y se exportan vía `lib/static/manual.ts` como `MANUAL_LINES y MANUAL_STOPS`.
4. `MANUAL_LINE_GEOJSON` mapea `descripcion → [url1, url2]` para que la UI pueda dibujar la polyline real (no el zigzag entre paradas sampleadas).

`lines.ts` y `stops.ts` mergean API + manual: si la API también lista una línea manual, gana la versión manual (más datos / polyline real).

2.3 GTFS

`scripts/generate-gtfs.ts` produce `public/gtfs/*.txt` (agency, routes, stops, shapes, trips, calendar, frequencies, stop_times) a partir de:

- `data/raw/lineas-transporte-urbano.csv` — empresas operadoras.
- `data/raw/paradas.geojson` — 10.081 paradas oficiales.
- `data/raw/recorridos.geojson` — 128 trazas `MultiLineString` (línea/sentido).
- `data/raw/frecuencias-2024.csv` — matriz hora × línea.

Es para **distribución a terceros** (validadores GTFS, Google Maps si algún día querés contribuir). El motor no lo consume.

> *No existe un GTFS público oficial de Mar del Plata. Google lo tiene pero no está liberado en datos abiertos. Generamos uno propio.*

---

3. El motor de routing (`lib/v2`)

3.1 Distancias: haversine

`lib/v2/spatial.ts:7` — fórmula clásica con radio terrestre `R = 6.371 km` :

```
a = sin²(Δφ/2) + cos(φ₁)·cos(φ₂)·sin²(Δλ/2)
d = 2·R·asin(√a)     (en metros)
```

Donde `φ` es latitud y `λ` longitud en radianes. La aproximación trata la Tierra como esfera (no elipsoide WGS84): error <0,5 % en distancias urbanas, sobra para nuestro caso.

Pre-filtro por bbox (`stopsWithinRadius` en spatial.ts:21): antes de correr haversine sobre 10k+ paradas se descartan las que están fuera de un rectángulo `[lat ± Δφ, lng ± Δλ]` , donde:

- `Δφ = radius / 111.320 m` (1° de latitud son ~111,32 km)
- `Δλ = radius / (111.320 · cos(lat))` (la longitud achica con la latitud)

Esto baja el costo de O(N) haversines a O(N) bbox-checks + O(K) haversines con K << N.

3.2 Grafo (`lib/v2/graph.ts`)

Nodos: todas las paradas (`STOPS`), indexadas por `stop.id` .

Aristas:

- **Bus** (dirigida): para cada bandera, conecta paradas consecutivas. Una línea con paradas `[A,B,C]` produce `A→B` y `B→C` . No es simétrica: si querés ir B→A tenés que tomar la otra bandera.
- **Walk** (bidireccional): entre paradas a ≤ 250 m de distancia. Permite transbordos a pie entre líneas distintas que paran cerca.

Pesos en minutos:

- `WALK_M_PER_MIN = 83` (5 km/h, peatón promedio)
- `BUS_M_PER_MIN = 300` (18 km/h, velocidad media urbana de un colectivo considerando frenadas y semáforos)
- `peso = distancia_haversine / velocidad_efectiva`

Construcción: O(L·B·P) para aristas bus (L líneas × B banderas × P paradas por bandera) + O(N^2) bbox-prefiltrado para aristas walk. Memoizado en módulo (`cached: Graph | null`): se construye una vez por proceso, reutilizado en todas las búsquedas.

3.3 Dijkstra con penalización de transbordos (`lib/v2/dijkstra.ts`)

Dijkstra estándar con min-heap (binario simple, suficiente para |V| ≈ 10k).

Lo que lo distingue:

1. **Múltiples puntos de partida y llegada con costo de caminata previo/ posterior.** Recibe:
   - `starts: { stopId, walkMts, walkMin }[]` — paradas cercanas al origen. El "costo inicial" del nodo no es 0, sino el tiempo de caminar hasta esa parada.
   - `ends: Map<stopId, { walkMts, walkMin }>` — paradas cercanas al destino, con su costo final de caminata.
2. **Penalización de transbordo** (`TRANSFER_PENALTY_MIN = 5`): cuando una arista bus es de una línea distinta a la que veníamos (`currentLine`), el peso suma 5 minutos extra (espera estimada al cambiar de bondi):

   ```
   if (e.type === "bus" && currentLine !== null && currentLine !== e.line) {
       cost += TRANSFER_PENALTY_MIN;
   }
   ```
3. **Reconstrucción y agrupación**: el path se rearma desde el destino siguiendo `prev` . Aristas bus consecutivas de la misma línea/bandera se colapsan en un solo paso `bus` con `stops += 1` (mostrar "subí en X, bajaste en X+12 paradas").
4. **Poda con `bestEnd`** : en cuanto se encuentra un nodo destino con costo total `T` , se descartan expansiones cuyo `dist[u] >= T` (no van a poder mejorar).

3.4 Búsqueda de itinerarios (`lib/v2/route-finder.ts`)

Dos modos:

- **`findRoutes`** — itinerarios de **una sola línea**: para cada línea cuyas banderas tocan tanto una parada cerca del origen como una cerca del destino, calcula el mejor par `(boardStop, alightStop)` . No usa Dijkstra, es una doble iteración. Más rápido, sin transbordos.
- **`findBestRoute`** — usa el Dijkstra completo. Devuelve el itinerario de menor tiempo total considerando hasta N transbordos.

La UI de `/v2/como-llego` corre **ambos en paralelo** y muestra primero los de una línea (suelen ser preferibles por simplicidad) y, si el de Dijkstra es significativamente más rápido, también lo ofrece.

3.5 Refinamiento peatonal con OSRM

Las caminatas que da Dijkstra son **distancia haversine / 83 m/min** — estimación en línea recta. Cuando el usuario abre el detalle de una parada, hacemos una request a OSRM (`profile=foot`) con sus coordenadas y las de la parada, y reemplazamos esa estimación por la distancia/duración real siguiendo la red de calles, además de obtener la polyline para dibujar el camino en el mapa.

OSRM corre **self-hosted** detrás de `osrm.aeterna.red` (Traefik + Cloudflared) para no depender del demo público.

---

4. ETA en vivo sin API muni

4.1 GPS crowdsourced via Telegram

Un bot recibe `live_location` de pasajeros que comparten el viaje y eligen qué línea y bandera están tomando. El backend persiste:

- `bondi.bus_locations` — última posición conocida por sesión.
- `bondi.bus_locations_history` — historial completo (para velocidad media).

Schema clave (de `db/migrations/002_bondi_bus_locations.sql`):

```
bus_locations(session_id PK, linea, ramal, lat, lng,
              velocity_kmh, avg_velocity_kmh, last_seen_at)
```

4.2 Velocidad

Para cada nuevo punto:

1. **Velocidad instantánea** (`server/src/routes/telegram.ts:126`): `v = (haversine(prev, actual) / Δt) · 3.6` km/h.
2. **Filtro de paradas**: si `v < MIN_KMH_FOR_AVG = 2` km/h, se descarta del cálculo de la media (asumimos que el bondi está parado en un semáforo o la persona bajó momentáneamente la pantalla).
3. **Mediana sobre ventana** (`telegram.ts:144`): tomamos los últimos `AVG_WINDOW = 8` puntos válidos y calculamos su **mediana** (no promedio: robusta a outliers como un GPS que saltó). Esa es `avg_velocity_kmh`.

4.3 Distancia bondi → parada del usuario

`server/src/routes/lineas.ts:60` — `distanceToStopByRoute()`:

1. Localiza la parada de la bandera más cercana al bondi (haversine sobre las paradas de esa bandera).
2. Suma haversines entre paradas consecutivas desde la del bondi hasta la parada destino del usuario:

   ```
   dist = Σ haversine(parada_i, parada_{i+1})
        para i desde la parada_del_bondi hasta parada_usuario − 1
   ```

Esta es una aproximación: usa la distancia entre paradas como proxy del recorrido real. No es la polyline exacta del trayecto pero, para banderas con paradas cada ~300 m, el error es <5 %.

4.4 ETA: dos modos

`server/src/routes/lineas.ts:298`:

```
const useReal = r.avg_velocity_kmh !== null && r.avg_velocity_kmh >= 5;
const kmh    = useReal ? r.avg_velocity_kmh : FALLBACK_KMH;  // 18 km/h
const etaMin = Math.max(0, Math.round((distMts / 1000 / kmh) * 60));
```

- **`gps_route`**: hay datos de velocidad real ≥ 5 km/h → usamos esa velocidad. ETA refleja tráfico real, paradas largas, etc.

- `gps_fallback`: no hay datos suficientes → usamos 18 km/h (velocidad urbana promedio). El usuario ve el ETA igual, con un indicador visual de que es una estimación.

El threshold de 5 km/h evita usar como "velocidad real" datos viciados por estar parado mucho tiempo en un semáforo.

---

5. Backend

5.1 Stack

- **Hono** (`@hono/node-server`) en `server/` — más liviano que Express, se exporta el handler como tipo `Hono.Hono` para tener tipos compartidos en el cliente.
- **Postgres** vía `pg` directo (no ORM, queries explícitas). Schema: `bondi.*` .
- **Migraciones manuales** en `db/migrations/*.sql` (sin Prisma, sin Drizzle): cada archivo se aplica `psql < archivo.sql` en orden.

5.2 Auth

JWT HS256 (`jose`) firmado con `JWT_SECRET` . Cookie `httpOnly` , `Secure` , `SameSite=None` (cross-site, porque el frontend en Vercel y el backend en otro dominio).

```
POST /auth/login   { email, password } → set-cookie + { user }
GET  /auth/me      → { user } | 401
POST /auth/logout  → unset cookie
```

Password hashing: `bcryptjs` con cost 10. Tabla `bondi.users(email PK, password_hash, name, ...)` .

5.3 Push notifications

VAPID con `web-push` . Flow:

1. Cliente genera subscription via `pushManager.subscribe()` con la `NEXT_PUBLIC_VAPID_PUBLIC` .
2. POST a `/push/subscribe` que la persiste en `bondi.push_subscriptions` .
3. Cuando un evento dispara (rutina cumple condición), un job en el backend manda `webpush.sendNotification(sub, payload)` .

Los workers de la rutina viven en `server/src/jobs/` .

5.4 Rutinas

Tabla `bondi.routines(user_id, kind, params, active)` . Tres tipos implementados (o planeados):

- `arrival_watch` — "avisame cuando el bondi 541 esté a < 3 min de la parada X entre lunes y viernes 18-19h".
- `daily_summary` — "todos los días a las 8 mandame un resumen de mi línea habitual".
- `nearby_alert` — "si hay un bondi a < 500 m de mí en horario laboral, avisame".

(Hay UI en `/v2/rutinas` para activar/desactivar; los jobs específicos están parcialmente implementados.)

---

6. Frontend

6.1 Páginas v2 (App Router)

- `/v2` — home con saludo y resumen (**actualmente con datos demo hardcodeados**, ver §8).
- `/v2/buscar` — búsqueda full-text de líneas y paradas.
- `/v2/linea/[codigo]` — detalle de línea: tabs por bandera, mapa con polyline y paradas clickeables, lista de paradas.
- `/v2/parada/[id]` — detalle de parada: arribos en vivo (GPS) + arribos muni, mapa con caminata desde el usuario, botón favorito.
- `/v2/como-llego` — origen (GPS o pin manual) + destino, devuelve itinerarios de una línea y multi-línea.
- `/v2/favoritos` — lista de paradas favoritas.
- `/v2/rutinas` — gestión de rutinas push.
- `/v2/login` — auth.

6.2 Mapas

Leaflet via `react-leaflet`. Todos los componentes mapa van envueltos en un wrapper que hace `dynamic(() => import("./Inner"), { ssr: false })` porque Leaflet toca `window` en import.

Tile provider: CartoDB Light (gratis, sin auth). Marcadores con `divIcon` (HTML inline, sin imágenes externas).

6.3 Datos del cliente

- **SWR** para fetching con cache + revalidation.
- **Optimistic updates** en favoritos/rutinas: el toggle aplica el cambio en el cache antes de que el backend responda.
- **Service Worker** (`public/sw.js`) maneja push events y abre la URL asociada al click en la notificación.

---

7. Plan de despliegue por PRs

> ***Contexto**: Mati gatekeepea `main`. Pidió **no mergear todo junto**, dividir por feature para que el rollout sea incremental y rollback-friendly. El PR #5 actual (rama `feat/v2-as-home`) tiene todo bundled — hay que decomponerlo.*

Estrategia

Cada PR debe:

- Compilar y pasar lint/typecheck **standalone** (cherry-pickeable a `main`).
- Tener una sola feature visible al usuario (o ser puramente backend/infra).
- Documentar en su descripción **qué activa** y **qué no toca**.

Orden propuesto

#	PR	Depende de	Visible al usuario
1	Backend infra (Postgres + Hono + Auth)	—	No (backend desplegado, cookie funciona)
2	Motor v2 + páginas `/v2/buscar` `/v2/linea` `/v2/parada` `/v2/como-llego`	1	Sí: rutas `/v2/*` accesibles, **home sigue siendo v1**

3	Login + favoritos + rutinas (UI y backend, sin push)	2	Sí: login en `/v2/login`, favoritos persistidos
4	Push notifications (VAPID + service worker + jobs)	3	Sí: las rutinas empiezan a notificar
5	GPS Telegram (bot + endpoint + tabla bus_locations)	1	No directo (el bot acepta mensajes)
6	Arribos en vivo + arribos muni en `/v2/parada`	2, 5	Sí: panel `LiveArrivals` con ETAs
7	Línea 221 (catálogo manual + GeoJSON)	2	Sí: `/v2/linea/221` aparece y funciona
8	Mapa de recorrido + paradas clickeables	2 (idealmente 7 también)	Sí: mapa en `/v2/linea/[codigo]`
9	Origen manual en `/v2/como-llego`	2	Sí: pin manual, GPS deja de ser obligatorio
10	Restaurar v1 como home + limpiar demo hardcodeado	2 (cualquier orden con 3+)	Sí: cambia la home
11	Docs + OSRM expuesto	— (paralelizable)	No

Commits "puente" (build fixes, tsconfig, suspense): se rebasean dentro del PR donde introducen el problema, no van separados.

Mapa commit → PR (estado actual)

```
8d55cfe (feat: server)              → PR 1
a13c447 (feat: dockerización)        → PR 1
0e7c153 (feat: v2 + lib/v2)          → PR 2 (hay que extraer lo de home v2)
991d7a5 (feat: login + fav + rut)    → PR 3 + PR 4 (separar push)
4d603a8 (fix: build/Suspense)        → rebasear dentro de PR 3
b2b685e (feat: GPS + arribos muni)   → PR 5 + PR 6 (separar)
a99fcba (docs + OSRM traefik)        → PR 11
152e2f1 (origen manual + 221 + mapa) → PR 7 + PR 8 + PR 9 (separar)
```

Los commits con "+" requieren `git rebase -i` interactivo (esto se hace manualmente, no por agente — riesgo de perder cambios).

---

8. Limpieza pendiente

8.1 Restaurar v1 como home

Decisión del producto: la home vuelve a ser el formulario step-by-step clásico (`app/v1/page.tsx` y compañía: `HomeClient` , `HomeIntro` , `LandingPage`). La razón: cambio menos brusco para los usuarios actuales.

Trabajo:

1. `app/page.tsx` deja de redirigir a `/v2`. Renderiza el contenido actual de `app/v1/page.tsx`.
2. Las rutas `/v2/*` siguen accesibles (buscar, parada, línea, como-llego, login, favoritos, rutinas).
3. El formulario v1 **debe usar el motor nuevo** (`lib/v2/route-finder.ts`) en vez de cualquier llamado directo a la API muni que tuviera antes. Hay que auditar `HomeClient` y reemplazar los fetch a la muni por `findRoutes` / `findBestRoute`.

Va en **PR 10** (depende de PR 2 que ya tiene el motor).

8.2 Eliminar el `DemoUser` hardcodeado

El `lib/demo/DemoUserContext.tsx` y `lib/demo/data.ts` definen un usuario ficticio "Nahuel" con:

- `viajesEstaSemana: 10`
- `zonaHabitual: "Constitución"`
- Rutinas activas pre-cargadas
- Banner contextual con texto fijo

Lo usan:

- `app/v2/_components/Header.tsx` — saludo y resumen.
- `app/v2/_components/RoutineCard.tsx` — card de rutina activa.
- `app/v2/_components/ContextualBanner.tsx` — banner.

Trabajo (PR 10 también, junto con la home):

- Reemplazar `useDemoUser()` por `useAuth()` (sesión real) o por `null` si no hay login.
- `viajesEstaSemana`, `zonaHabitual` → calcularlos del backend (tabla de eventos de uso) o **ocultarlos** hasta que haya datos reales (preferido: no mostrar lo que no podemos sustentar).
- Rutinas → ya hay backend real (`/routines`). Cambiar `RoutineCard` para consumirlo via SWR.
- `ContextualBanner` → o se elimina, o se alimenta de una regla real (ej: "tu próxima rutina dispara en 12 minutos").
- Borrar `lib/demo/` por completo cuando nadie lo importa.

---

9. Glosario rápido

- **Bandera**: sentido de una línea (ida/vuelta o cabeceras). Una línea suele tener 2 banderas, a veces más si hay ramales.
- **Itinerario**: secuencia de pasos walk/bus/transfer que conecta dos ubicaciones.
- **Transbordo**: bajarse de una línea y subirse a otra (a veces caminando hasta una parada cercana).
- **ETA**: estimated time of arrival, tiempo estimado hasta que el bondi llega a una parada específica.
- **OSRM**: Open Source Routing Machine, motor de ruteo sobre OpenStreetMap. Lo usamos en perfil `foot` para caminatas reales.
- **VAPID**: Voluntary Application Server Identification, par de claves para autenticar el origen de las web push notifications.